



汇编语言

9 分支结构与循环结构

张青

ieqzhang@zzu.edu.cn

实验三 操作数寻址

- 1、 录入 exp0301.S，汇编连接后，用 GDB 单步调试，填写表 1。
- 2、 录入例 3-9，编译生成汇编语言程序和可执行程序，分析程序运行结果。用 GDB 单步调试，观察指令执行结果，填写表 2。
- 3、 编写汇编语言程序 exp0303.S，在数据段定义 4 个双字数据，分别采用寄存器间接寻址和寄存器相对寻址两种方法读取元素，调用 dispsid 显示元素的值。

教学内容



- 转移指令
- 分支结构设计

教材章节：5.1

慕课：7

- ▶ 指令寻址方式
- ▶ 无条件转移指令
- ▶ 条件转移指令
- ▶ 设置条件码

- 转移指令实现循环结构
- 循环指令实现循环结构

教材章节：5.2

慕课：8

- ▶ 循环指令
- ▶ 计数控制循环
- ▶ 条件控制循环

学习目标

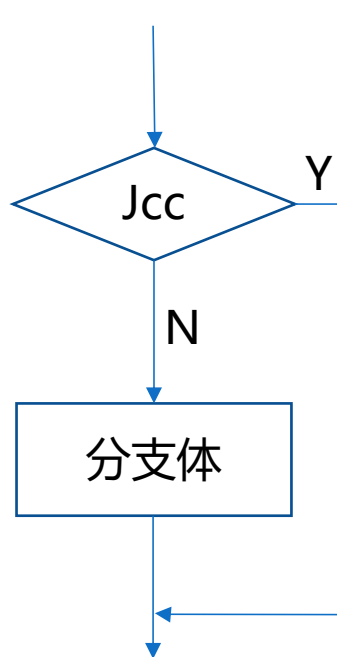


- 能说出转移指令的功能和格式
- 会正确判断转移发生的条件
- 会正确使用转移指令实现单分支和双分支结构
- 会阅读分析分支结构程序的功能、清楚运行结果
- 记忆循环指令loop的功能
- 会阅读分析循环结构程序的功能、运行结果
- 会用gdb查看寄存器和内存
- 能说出程序开发过程中出现问题的原因并解决
- 养成良好的编程习惯，养成细心、严谨的工作态度
- 养成团队合作，互相帮助的精神

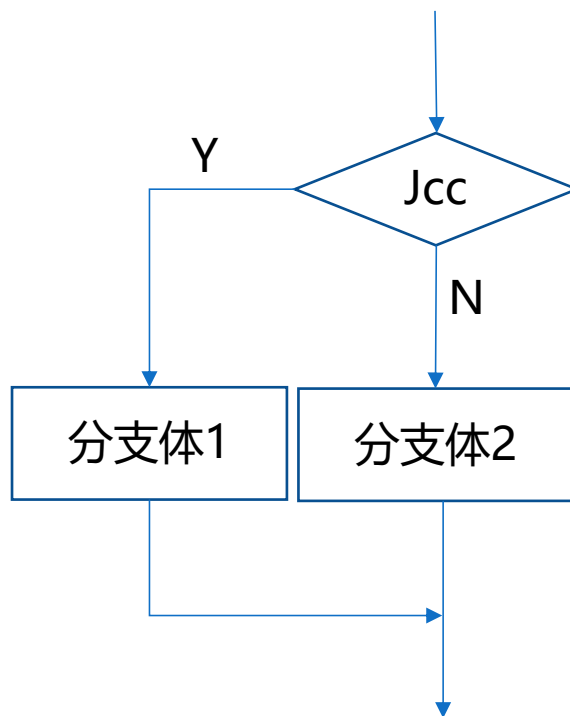


分支结构设计

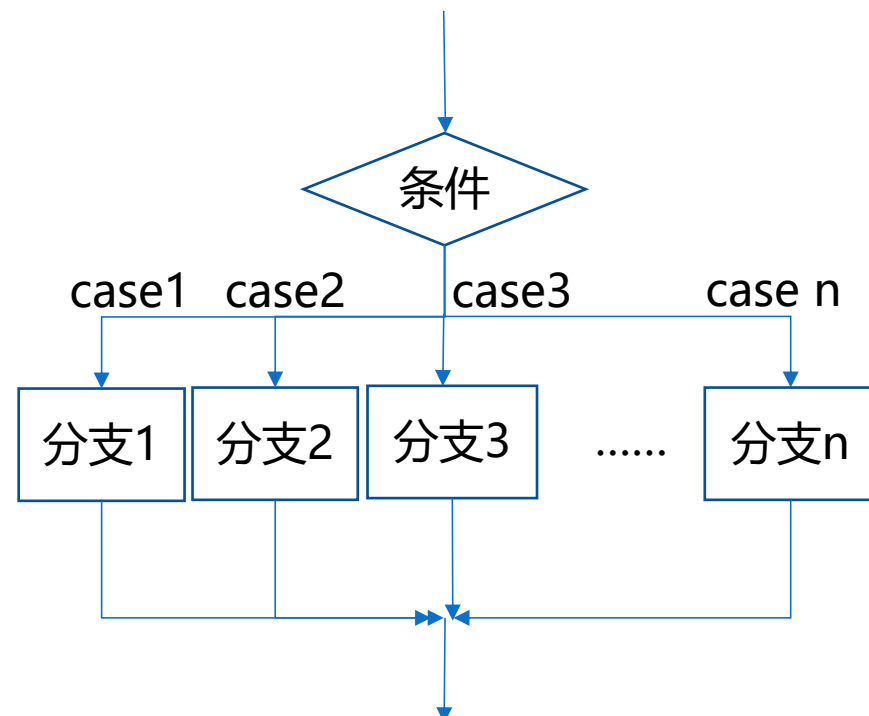
三种分支结构



(a)单分支



(b)双分支



(c)多分支

例5-1: diffabs函数



```
int diffabs(int x, int y){
int t;
if(x >= y)
{
t=x-y;
goto L1;
}
t=y-x;
L1:
return t;
}
```

```
diffabs: #双分支
#ecx: x, edx: y
    cmp ecx, edx
    jge .L4
    mov eax, edx
    sub eax, ecx
    .....
.L1: #汇合
    ret
.L4:
    mov eax, ecx
    sub eax, edx
    jmp .L1
```

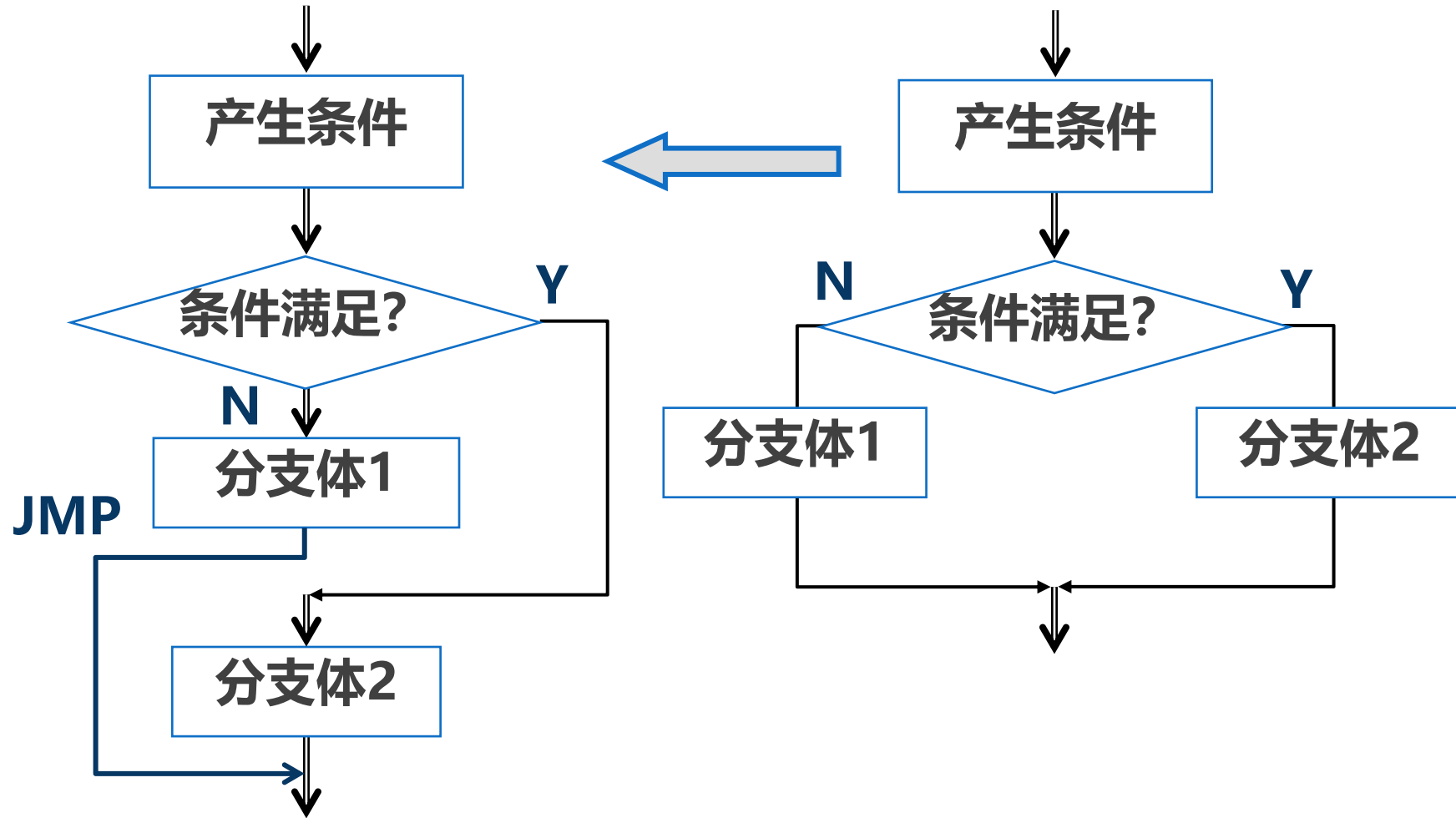
diffabs: #单分支

.....
mov eax, ecx
sub eax, edx

cmp ecx, edx
jge .L4
mov eax, edx
sub eax, ecx

.....
.L4:
mov eax, ecx
sub eax, edx
.L1: ret

双分支结构的流程图



〔例5-2〕 求x的绝对值



```
int iabs(int x){  
    if(x<0) x=-x;  
    return x;  
}
```

#O0优化, 单分支

```
mov DWORD PTR 16[rbp], ecx # x在ecx中  
cmp DWORD PTR 16[rbp], 0  
jns .L2  
neg DWORD PTR 16[rbp]  
.L2: mov eax, DWORD PTR 16[rbp]
```

#人工优化, 单分支

```
mov eax, ecx # x在ecx中  
test ecx, ecx  
  
jns .L2 # x不是负数  
.L3: neg eax # x是负数  
L2: ret
```

#Og优化, 双分支

```
mov eax, ecx # x在ecx中  
test ecx, ecx  
js .L3 # x是负数  
.L2: ret #分支1, 分支汇合  
.L3: neg eax #分支2  
jmp .L2 #分支2结束
```

〔例5-3〕 字母判断程序



```
char Uchar(char c){  
if(c>='A' && c<='Z')  c=c+32;  
return c;  
}
```

```
#Og优化, 单分支  
mov  eax, ecx      # c在ecx中  
lea  edx, -65[rcx] # 65是 'A' 的ASCII码值  
cmp  dl, 25  
ja   .L2           # dl大于25, 不是大写字母  
lea  eax, 32[rcx]  
# dl小于等于25, 是大写字母, 转换为小写字母  
.L2:
```

```
#人工编写汇编指令, c在al中  
cmp al, 'A'      #与大写字母A比较  
jb done         #比大写字母A小, 不是大写字母, 转移  
cmp al, 'Z'      #与大写字母Z比较  
ja done         #比大写字母Z大, 不是大写字母, 转移  
or al, 0x20      #转换为小写  
done:
```

〔例5-4〕有符号数运算溢出程序



```
//eg0504.c
int sub_of(int x, int y, int *diff){
    *diff=x-y;
    return 0;
}

int main(){
    int d,f;
    f=sub_of(1234567890,-999999999,&d);
    printf("x-y = %d = %u = %x",d,d,d);
}
```

```
sub_of: #eg0504.s
    #x在ecx, y在edx, diff在r8
    sub ecx, edx
    mov DWORD PTR [r8], ecx
    mov eax, 0 #返回值在eax
    ret
```

$x-y = -2060399407 = 2234567889 = 8530ccd1$

溢出!

〔例5-4〕 有符号数运算溢出程序



```
//eg0504b.c
int sub_of(int x, int y, int *diff){
    *diff=x-y;
    return 0;
}

int main(){
    int d,f;
    f=sub_of(1234567890,-9999999999,&d);
    if(!f)
        printf("x-y = %d = %u = %x",d,d,d);
    else
        printf( "溢出");
}
```

```
sub_of: #eg0504b.S
    #x在ecx, y在edx, diff在r8
    sub ecx, edx
    mov DWORD PTR [r8], ecx
    jno L3          #没有溢出, 返回值为0
    L1:mov eax,1    #有溢出, 返回值为1
    jmp L2
    L3: mov eax, 0   #返回值在eax
    L2: ret
```

gcc -o eg0504b ./progs/eg0504b.S

溢出

修改汇编语言程序提高了C语言程序的可靠性。

gcc -S -Og -masm=intel -o eg0504b.s ./progs/eg0504b.c

〔例5-5〕 数组访问程序



假设有8个信息（字符串），编程显示指定的信息。具体功能是：

- （1）提示输入数字，并输入数字；
- （2）判断数字是否在规定的范围内，不在范围内、提示错误信息；
- （3）输入在规定的范围内，显示数字对应的信息，退出。

```
char* msgs[]={"Chapter 1: Fundamentals\n",  
              "Chapter 2: Data Representation\n",  
              "Chapter 3: x86 Intruction Set Architecture\n",  
              "Chapter 4: Basic Instructions\n",  
              "Chapter 5: Program Structure\n",  
              "Chapter 6: ARM Intruction Set Architecture\n",  
              "Chapter 7: ARM Assembly Language Programming\n",  
              "Chapter 8: Assembly Language Applications\n"};
```

〔例5-5〕 数据段



```
.section .rdata,"dr"  #只读数据段
.LC3: .ascii "Chapter 1: Fundamentals\12\0"
.LC4: .ascii "Chapter 2: Data Representation\12\0"
.LC5: .ascii "Chapter 3: x86 Intruction Set Architecture\12\0"
.LC6: .ascii "Chapter 4: Basic Instructions\12\0"
.LC7: .ascii "Chapter 5: Program Structure\12\0"
.LC8: .ascii "Chapter 6: ARM Intruction Set Architecture\12\0"
.LC9: .ascii "Chapter 7: ARM Assembly Language Programming\12\0"
.LC10: .ascii "Chapter 8: Assembly Language Applications\12\0"
.data                #数据段
msgs: .quad .LC3      #字符串地址
      .quad .LC4
      .quad .LC5
      .quad .LC6
      .quad .LC7
      .quad .LC8
      .quad .LC9
      .quad .LC10
```

〔例5-5〕 主程序

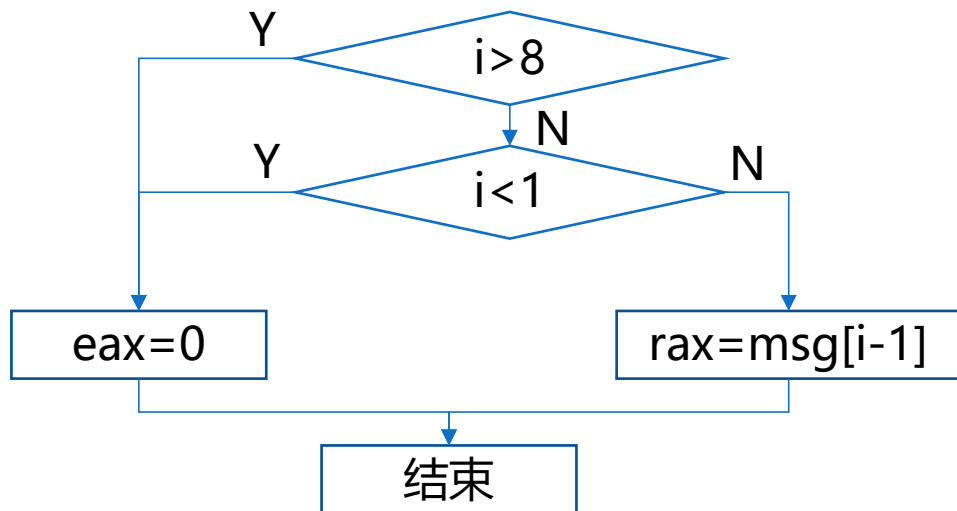


```
char* print(char* msgs[], int i);  
int main(){  
    int input;  
    char* pc;  
    scanf("%d",&input);           //输入编号  
    pc=print(msgs,input);         //获取编号对应字符串地址  
    if(pc) printf("%s",pc);       //找到字符串  
    else printf("input error");   //没有找到  
    return 0;  
}
```

〔例5-5〕 print函数



```
char* print(char* msgs[], int i){  
    if(i>8 || i<1)  
        return 0;  
    else  
        return msgs[i-1];  
}
```

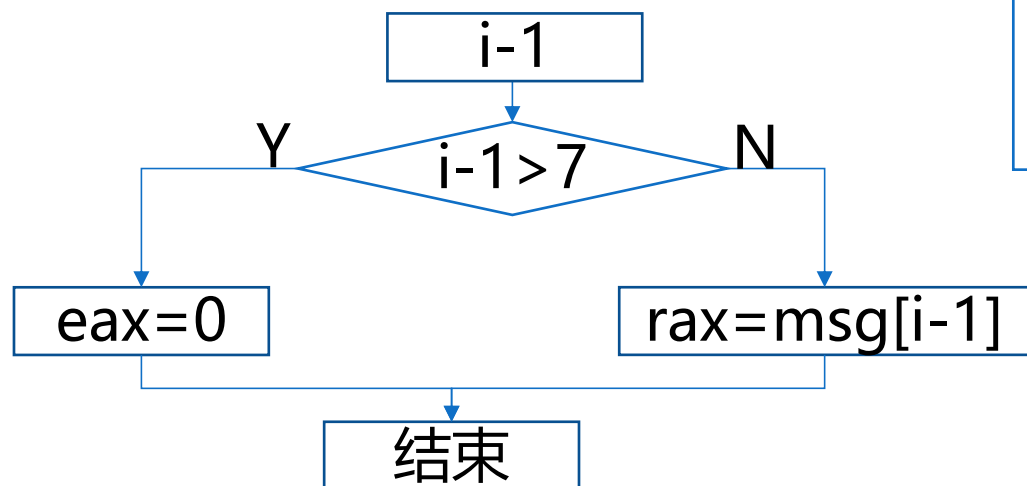


```
#人工汇编, rcx=msgs, edx=i  
cmp edx,8      # 判断edx的范围  
ja L1          # 大于8  
cmp edx, 1     # 小于1  
jb L1  
dec edx  
movsx rdx, edx  
mov rax, QWORD PTR [rcx+rdx*8]  
ret            # jmp L2  
L1: mov eax,0  
L2: ret
```


〔例5-5〕 优化函数print



```
char* print(char* msgs[], int i){  
    if(i>8 || i<1)  
        return 0;  
    else  
        return msgs[i-1];  
}
```



```
print: #-Og优化, rcx=msgs, edx = i  
    lea    eax, -1[rdx]    #eax=i-1  
    cmp    eax, 7  
    ja     .L3  
    movsx  rdx, edx  
    mov    rax, QWORD PTR -8[rcx+rdx*8]  
.L1:    ret                #函数返回  
.L3:    mov    eax, 0      #eax=返回值  
    jmp    .L1
```

i=0时, eax=?

〔例5-6〕 地址表程序

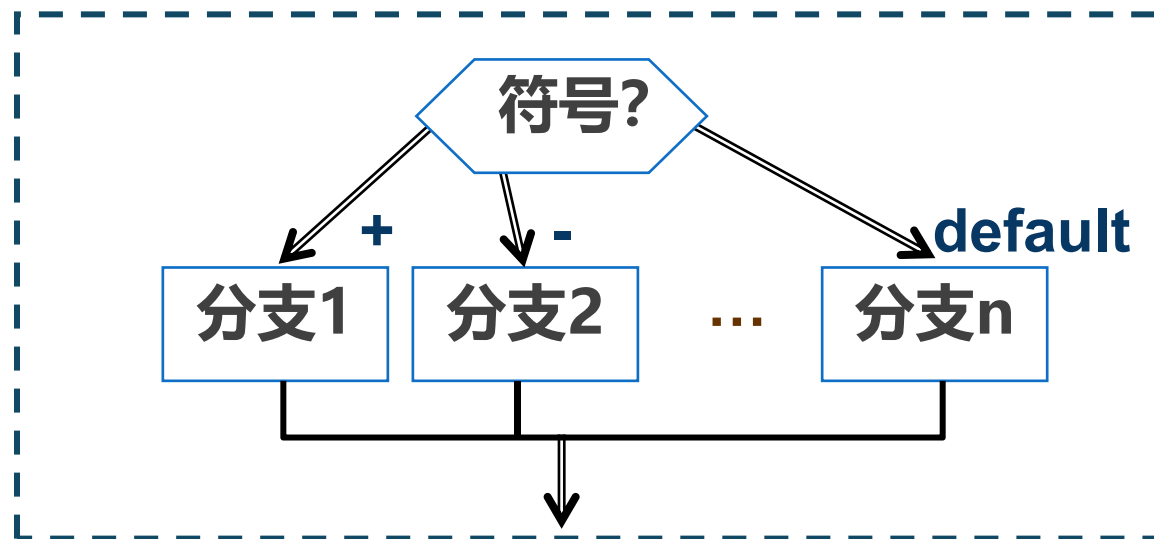


编制一个简单的四则运算计算器。该程序根据输入的规定符号实现两个有符号数加、减、乘、除、移位等运算。输入其他字符则提示非法并退出程序。

```
int caculator(char n, int x, int y ,
int *result){
int f=0;
switch(n){
case '+':
    *result = x+y;
    break;
case '-':
    *result = x-y;
    break;
case '*':
    *result = x *y;
    break;
```

```
case '/':
    *result =x/y;
    break;
case '<':
    *result = x<<1;
    break;
case '>':
    *result = y>>2;
    break;
default:
    f=1;
    break;}
return f;}
```

〔例5-6〕 汇编语言实现思路



- 地址表实现方法:
 - 根据符号计算目标地址
 - 采用间接寻址

〔例5-6〕 地址表汇编语言程序中分支定义



```
.text # O2优化, edx=x, r8d=y, r9=result, eax=f
.....
.L11:  mov     eax, 1      # default
      ret

.....
.L3:   sar     r8d, 2      # case '>' (60) , r8d=y
      xor     eax, eax
      mov     DWORD PTR [r9], r8d
.L13:  ret
.L9:   imul    r8d, edx    # case '*' (42) , r8d=y
      xor     eax, eax
      mov     DWORD PTR [r9], r8d
      jmp     .L13

.....
.L8:   add     r8d, edx    # case '+' (43) , r8d=y
      xor     eax, eax
      mov     DWORD PTR [r9], r8d
      jmp     .L13
```

〔例5-6〕 地址表汇编语言程序中分支定义



```
.L7:  sub    edx, r8d  # case  '-'  (45) , r8d=y
      xor    eax, eax
      mov    DWORD PTR [r9], edx
      jmp    .L13
      .....
.L6:  mov    eax, edx  # case  '/'  (47) , r8d=y
      cdq
      idiv   r8d
      mov    DWORD PTR [r9], eax
      xor    eax, eax
      jmp    .L13
      .....
.L5:  add    edx, edx  #case  '<'  (62) ,edx=x
      xor    eax, eax
      mov    DWORD PTR [r9], edx
      jmp    .L13
```

〔例5-6〕地址表汇编语言程序中偏移量定义



```
.section .rdata,"dr"
.L4:  .long  .L9-.L4      # 标号.L9距离.L4的位移量
      .long  .L8-.L4      # 标号.L8距离.L4的位移量
      .long  .L11-.L4     # 标号.L11距离.L4的位移量
      .long  .L7-.L4      # 标号.L7距离.L4的位移量
      .long  .L11-.L4     # 标号.L11距离.L4的位移量
      .long  .L6-.L4      # 标号.L6距离.L4的位移量
      .....
      .long  .L5-.L4      # 标号.L5距离.L4的位移量
      .long  .L11-.L4     # 标号.L11距离.L4的位移量
      .long  .L3-.L4      # 标号.L3距离.L4的位移量
      .....
```

>	.L3-.L4	+80
	.L11-.L4	
<	.L5-.L4	
		
/	.L6-.L4	
	.L11-.L4	
-	.L7-.L4	+12
	.L11-.L4	+8
+	.L8-.L4	+4
*	.L9-.L4	.L4

ASCII码表



ASCII码	字符	ASCII码	字符	ASCII码	字符	ASCII码	字符
00H	NUL	20H	SP	40H	@	60H	`
01H	SOH	21H	!	41H	A	61H	a
02H	STX	22H	"	42H	B	62H	b
03H	ETX	23H	#	43H	C	63H	c
04H	EOT	24H	\$	44H	D	64H	d
05H	ENQ	25H	%	45H	E	65H	e
06H	ACK	26H	&	46H	F	66H	f
07H	BEL	27H	'	47H	G	67H	g
08H	BS	28H	(	48H	H	68H	h
09H	HT	29H	)	49H	I	69H	i
0AH	LF	2AH	*	4AH	J	6AH	j
0BH	VT	2BH	+	4BH	K	6BH	k
0CH	FF	2CH	,	4CH	L	6CH	l
0DH	CR	2DH	-	4DH	M	6DH	m
0EH	SO	2EH	.	4EH	N	6EH	n
0FH	SI	2FH	/	4FH	O	6FH	o
10H	DLE	30H	0	50H	P	70H	p
11H	DC1	31H	1	51H	Q	71H	q
12H	DC2	32H	2	52H	R	72H	r
13H	DC3	33H	3	53H	S	73H	s
14H	DC4	34H	4	54H	T	74H	t
15H	NAK	35H	5	55H	U	75H	u
16H	SYN	36H	6	56H	V	76H	v
17H	ETB	37H	7	57H	W	77H	w
18H	CAN	38H	8	58H	X	78H	x
19H	EM	39H	9	59H	Y	79H	y
1AH	SUB	3AH	:	5AH	Z	7AH	z
1BH	ESC	3BH	;	5BH	[	7BH	{
1CH	FS	3CH	<	5CH	\	7CH	
1DH	GS	3DH	=	5DH	]	7DH	}
1EH	RS	3EH	>	5EH	^	7EH	~
1FH	US	3FH	?	5FH	-	7FH	Del

〔例5-6〕地址表汇编语言程序



int caculator(char n, int x, int y, int *result);

```
.text
.....
caculator: #O2优化, ecx=n, edx =x, r8d=y, r9=result
.....#计算目标地址
sub    ecx, 42      # ecx=n-42, 42是 '*' 的ASCII码
cmp    cl, 20       #位移量=n- '*' , '>' - '*' =20
ja     .L11         # 大于20, 转至标号.L11(default)
lea     r10, .L4[rip] # r10 =.L4
movzx   ecx, cl     #零扩展cl到ecx
movsx   rax, DWORD PTR [r10+rcx*4] # rax=目标地址-.L4
add     rax, r10     # rax=目标地址
jmp     rax         # 绝对转移, rip=rax
```

>	.L3-.L4	+80
	.L11-.L4	
<	.L5-.L4	
		
/	.L6-.L4	
	.L11-.L4	
-	.L7-.L4	+12
	.L11-.L4	+8
+	.L8-.L4	+4
*	.L9-.L4	.L4

〔例5-6〕 人工修改简化汇编语言程序



```
.section .rdata,"dr"
.align 4
.L4: .quad .L9  # '*', 直接存目标地址
     .quad .L8  # '+'
     .quad .L11 # default
     .quad .L7  # '-'
     .quad .L11 # default
     .quad .L6  # '/'
     .quad .L11 # default
     .quad .L11 # default
     .....
     .quad .L11 # default
     .quad .L5  # '>'
     .quad .L11 # default
     .quad .L3  # '<'
```

>	.L3	+160
	.L11	
<	.L5	
		
	.L11	
/	.L6	
	.L11	
-	.L7	+24
	.L11	+16
+	.L8	+8
*	.L9	.L4

〔例5-6〕 人工修改简化汇编语言程序



calculator:

```
.....  
sub    ecx, 42  
cmp    cl, 20  
ja     .L11  
lea    r10, .L4[rip]  
movzx  ecx, cl  
mov    rax, QWORD PTR [r10+rcx*8]  
jmp    rax
```

>	.L3	+160
	.L11	
<	.L5	
	L11	
/	.L6	
	.L11	
-	.L7	+24
	.L11	+16
+	.L8	+8
*	.L9	.L4

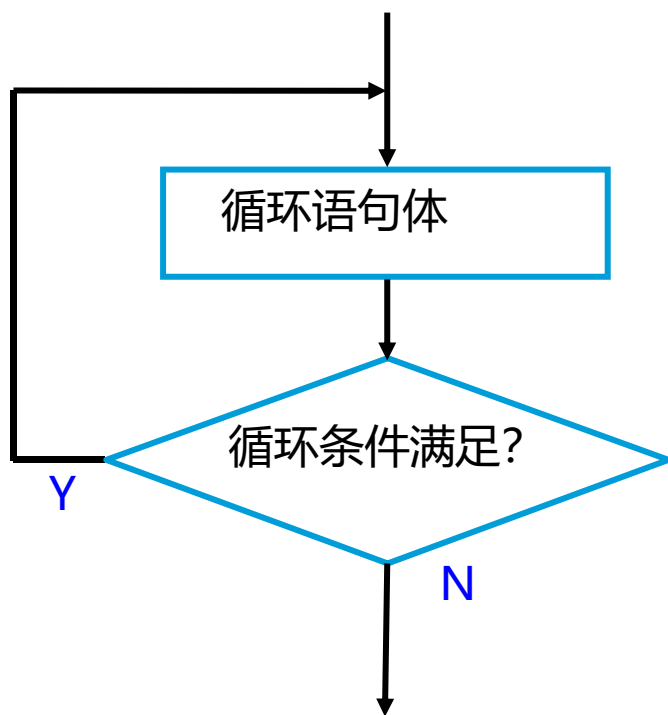
Windows系统如果采用优化参数-Og，switch语句用条件判断实现，而非地址表
Linux系统下用地址表实现



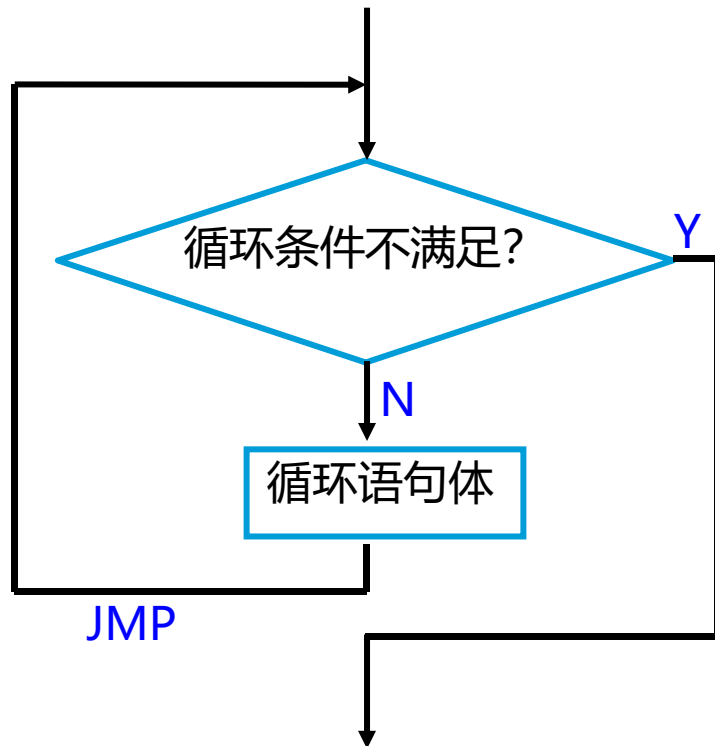
循环结构

- 三个部分组成：
 - 循环初始——为开始循环准备必要的条件，如循环次数、循环体需要的初始值等；
 - 循环体——重复执行的程序代码，其中包括对循环条件的修改等；
 - 循环控制——判断循环条件是否成立，决定是否继续循环
- “先判断、后循环”的循环程序结构
 - 对应高级语言的WHILE语句
- “先循环、后判断”的循环程序结构
 - 对应高级语言的DO语句
- 计数循环
 - 对应高级语言的for语句

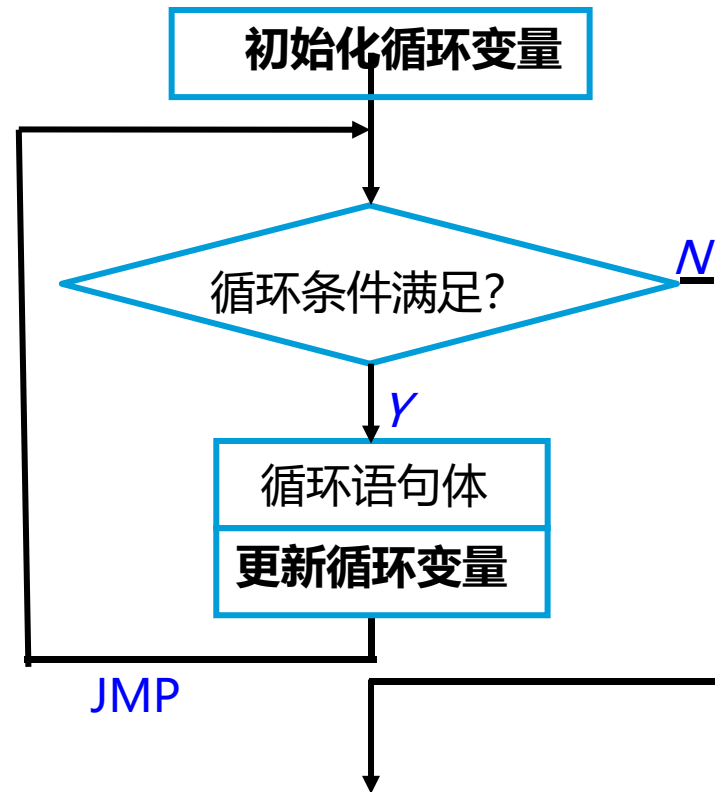
循环程序结构的流程图



(a) “先循环、后判断” 结构



(b) “先判断、后循环” 结构



(c) for 循环结构

用转移指令实现循环结构



```
int find(int v[3], int target) {  
    int i;  
    for (i = 0; i < 3; i++) {  
        if (v[i] == target) {  
            goto found;  
        }  
    }  
    return -1;  
  
found:  
    return i;  
}
```

```
find: #rcx=数组首地址v, edx=target  
      mov    eax, 0 #eax=i  
      jmp    .L6  
      .L10:  add    eax, 1  
.L6:   cmp    eax, 2  
      jg     .L9  
      movsx  r8, eax  
      cmp    DWORD PTR [rcx+r8*4], edx  
      jne    .L10  
      jmp    .L5 #找到, eax=下标  
.L9:   mov    eax, -1 #循环次数结束, 没找到  
.L5:   ret
```